DevOps & GitOps

Méthodologie, pratiques et outils — du débutant à l'expert

DÉBUTANT | INTERMÉDIAIRE | GITOPS | EXPERT

NZEMBOUONG Patrick
Ingénieur DevOps & Kubernetes

Parcours du cours

1

Débutant

Pourquoi DevOps, culture, CI/CD, outils. TP : premier pipeline

2

Intermédiaire

IaC, conteneurs, déploiements, observabilité, DevSecOps, DORA. TP : Docker, Kubernetes, Terraform

3

GitOps

Principes, push vs pull, Argo CD / Flux, workflow. 2 TP : Argo CD, dérive

4

Expert

Secrets, multi-cluster, policy as code, supply chain, platform engineering. TP : secrets, politiques, canary

01

Les bases du DevOps

Comprendre le problème, la culture et le pipeline

DÉBUTANT

Le problème : Dev contre Ops



Les Développeurs

Veulent livrer des nouveautés vite.

Objectif : le changement

Le mur de la confusion

« Ça marche sur ma machine ! »



Les Opérations

Veulent un système stable.

Objectif : la stabilité

- Livraisons rares, grosses et risquées
- Reproches mutuels, peu de visibilité
- Délais de plusieurs semaines pour une mise en production

DÉBUTANT

Qu'est-ce que le DevOps ?

Une culture + des pratiques + des outils qui rapprochent Dev et Ops pour livrer plus vite, plus souvent et avec fiabilité.



Culture

Collaboration, responsabilité partagée



Automatisation

Build, tests, déploiement, infra



Mesure

Métriques, observabilité



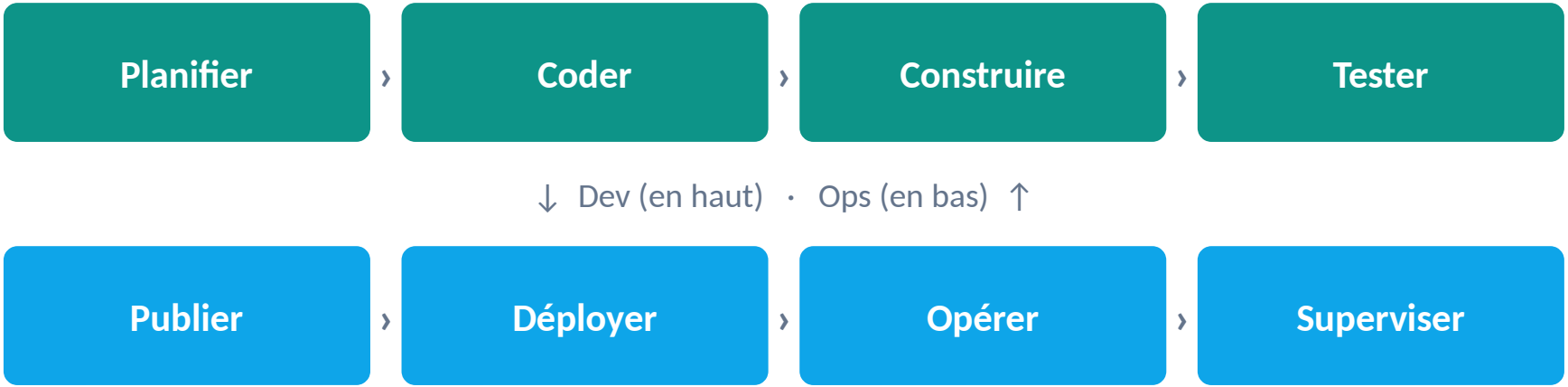
Partage

Connaissances, retours d'expérience

Modèle CALMS : Culture · Automation · Lean · Measurement · Sharing

DÉBUTANT

La boucle infinie DevOps



Feedback continu

Le monitoring en production alimente la planification : chaque boucle améliore la suivante.

DÉBUTANT

Les pratiques clés



Contrôle de version

Git pour tout le code



Intégration continue

Build + tests à chaque commit



Livraison continue

Toujours prêt à déployer



Infrastructure as Code

Infra décrite en fichiers



Supervision

Logs, métriques, alertes

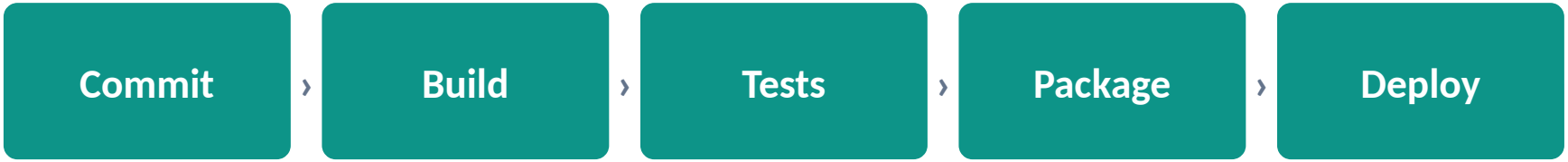


Feedback rapide

Boucles courtes

DÉBUTANT

Le pipeline CI/CD



Commit
Push sur Git déclenche le pipeline

Build
Compilation, dépendances

Tests
Unitaires, intégration, lint

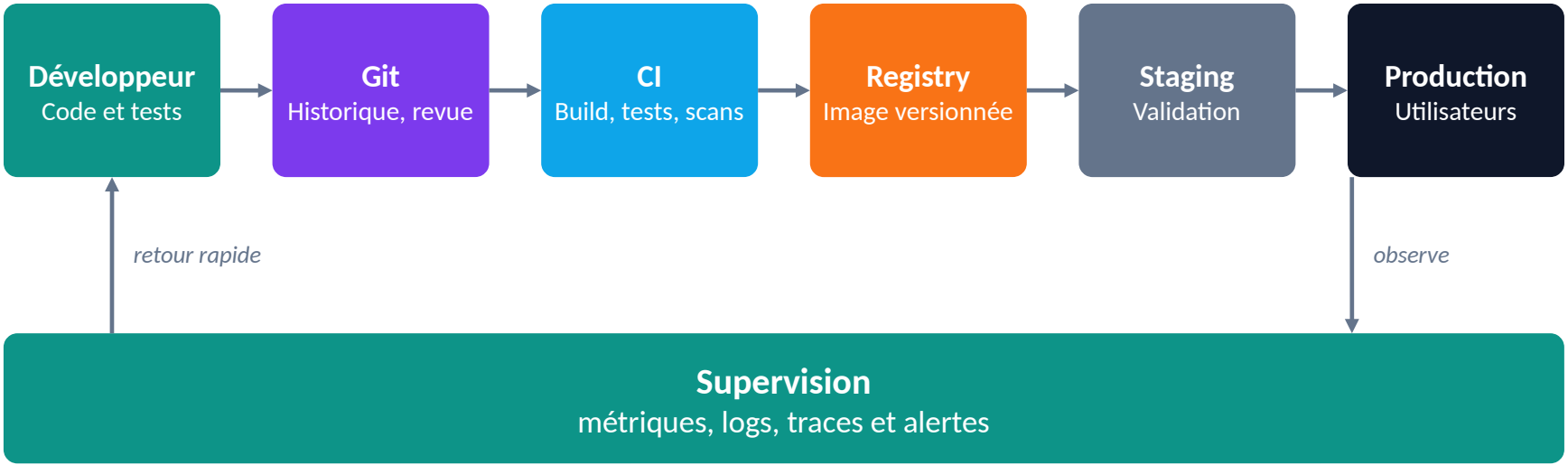
Package
Image Docker, artefact versionné

Deploy
Vers staging puis production

Règle d'or

Pipeline rapide (< 10 min), échec = on stoppe tout et on corrige d'abord. Le build est la responsabilité de toute l'équipe.

Schéma : chaîne de livraison DevOps



Chaque étape est automatisée ; chaque échec renvoie un retour rapide à l'équipe.

DÉBUTANT

CI, livraison continue, déploiement continu

CI

Intégration continue

Chaque commit est compilé et testé automatiquement. Objectif : détecter vite les erreurs.

CD

Livraison continue

Le code est toujours prêt à être déployé. Le déploiement en production reste un clic manuel.

CD

Déploiement continu

Chaque changement validé part en production automatiquement, sans intervention.

DÉBUTANT

Les outils par phase

Phase	Rôle	Exemples d'outils
Code	Versionner	Git, GitHub, GitLab, Bitbucket
CI/CD	Automatiser build & déploiement	GitHub Actions, GitLab CI, Jenkins
Conteneurs	Empaqueter	Docker, Podman, Buildah
Orchestration	Exécuter à l'échelle	Kubernetes, OpenShift
IaC	Décrire l'infra	Terraform, Ansible, Pulumi
Supervision	Observer	Prometheus, Grafana, ELK, Loki

DÉBUTANT

TP : premier pipeline CI

Durée
1 h 30

Objectif
Construire un pipeline GitHub Actions qui teste et empaquette une petite application.

Prérequis
Compte GitHub, Git installé, une appli simple (Node ou Python).

1. Créer un dépôt GitHub et y pousser l'application
2. Ajouter .github/workflows/ci.yml déclenché sur push et pull_request
3. Ajouter les étapes : checkout, installation, lint, tests
4. Casser volontairement un test, constater l'échec, puis corriger
5. Ajouter un badge de statut dans le README

Livrable : un dépôt avec un pipeline vert, et la capture d'un échec corrigé.

DÉBUTANT

Quiz niveau Débutant

1 **Que signifie DevOps ?**
A) Un langage de programmation B) Un outil de déploiement C) Une culture qui rapproche développement et exploitation

2 **Quel est l'objectif de l'intégration continue ?**
A) Compiler et tester automatiquement chaque commit B) Remplacer Git C) Déployer en production sans aucun test

3 **Quelle différence entre livraison continue et déploiement continu ?**
A) Aucune, ce sont deux noms du même terme B) Le déploiement continu envoie en production sans approbation manuelle C) La livraison continue n'exécute pas de tests

4 **Que faire quand le pipeline est rouge ?**
A) L'ignorer et continuer B) Désactiver les tests concernés C) Le corriger avant tout autre travail

5 **Que symbolise le « mur de la confusion » ?**
A) La séparation entre Dev (changement) et Ops (stabilité) B) Un pare-feu réseau C) Un outil de supervision

02

Pratiques intermédiaires

Infrastructure, conteneurs, déploiement, observabilité, sécurité

INTERMÉDIAIRE

Infrastructure as Code (IaC)

- Infra décrite en code, versionnée dans Git
- Reproductible : mêmes fichiers = même résultat
- Revue de code et historique des changements
- Déclaratif (Terraform) vs configuration (Ansible)

```
resource "aws_instance" "web" {
  ami           = "ami-0abc123"
  instance_type = "t3.micro"

  tags = {
    Name = "serveur-web"
    Env  = "prod"
  }
}

# terraform plan
# terraform apply
```

Conteneurs et orchestration

Docker

Empaquette l'application et ses dépendances dans une image portable et immuable.

Kubernetes

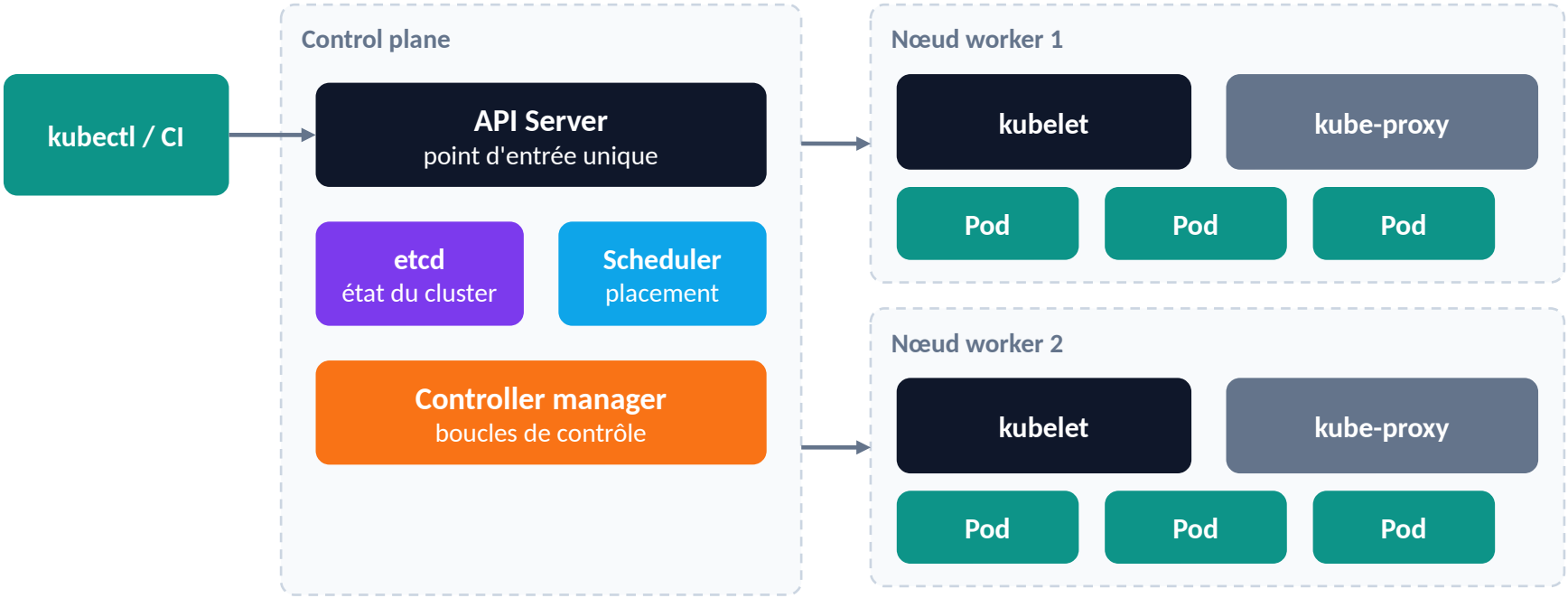
Déploie, met à l'échelle et répare automatiquement les conteneurs.

```
FROM node:20-alpine
WORKDIR /app
COPY . .
RUN npm ci --omit=dev
CMD ["node","server.js"]
```



Image → registre → pods → exposé via un service

Schéma : architecture Kubernetes



Les nœuds worker exécutent les conteneurs ; le control plane décide de l'état voulu.

INTERMÉDIAIRE

Stratégies de déploiement

Rolling update

Remplace les instances une par une.

+ simple, sans coupure

− rollback plus lent

Blue / Green

Deux environnements : on bascule le trafic d'un coup.

+ rollback instantané

− coût doublé

Canary

Nouvelle version pour 5 % du trafic, puis 25 %, 100 %.

+ risque minimal

− plus complexe

INTERMÉDIAIRE

Monitoring et observabilité

Métriques

CPU, latence, taux d'erreurs. Prometheus + Grafana.

Logs

Événements détaillés. Loki, ELK.

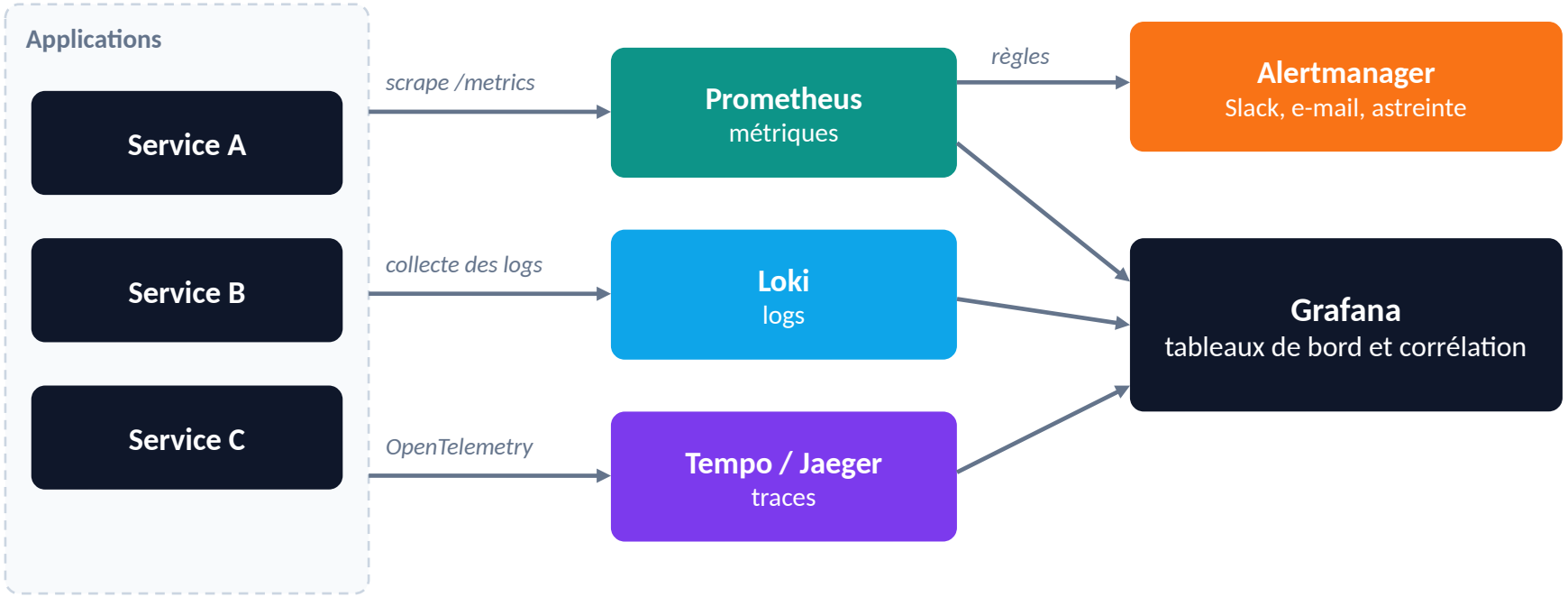
Traces

Parcours d'une requête entre services. OpenTelemetry, Jaeger.

Alerter sur les symptômes

Alerter sur ce que vivent les utilisateurs (SLO, erreurs, latence), pas sur chaque CPU à 80 %.

Schéma : pile d'observabilité



Trois sources de données, un seul point de lecture pour diagnostiquer.

DevSecOps : la sécurité dès le début

Code › **Build** › **Test** › **Release** › **Run**

SAST / lint
Analyse du code source

SCA
Dépendances vulnérables

Scan d'images
Trivy, Grype

DAST
Tests dynamiques

Runtime
Falco, politiques

Shift-left

Détecter tôt = corriger moins cher. Les scans tournent dans le pipeline.

Secrets

Jamais de mot de passe dans Git. Utiliser un coffre (Vault, gestionnaire cloud).

Mesurer le succès : les métriques DORA

Vitesse

À la demande

Fréquence de déploiement

Vitesse

< 1 jour

Délai de livraison

Stabilité

0 – 15 %

Taux d'échec des changements

Stabilité

< 1 heure

Temps de restauration

Repères « Elite » du rapport DORA

INTERMÉDIAIRE

TP : Docker, Kubernetes et Terraform

Durée
3 h

Objectif
Conteneuriser une application, la déployer sur un cluster local et décrire son infrastructure en code.

Prérequis
Docker, kind ou minikube, kubectl, Terraform.

1. Écrire un Dockerfile et construire l'image : docker build -t shop:1.0 .
2. Créer un cluster local : kind create cluster
3. Écrire deployment.yaml et service.yaml, puis kubectl apply -f .
4. Provisionner avec Terraform (fournisseur docker ou kubernetes) : plan, apply, destroy
5. Tester une mise à jour progressive : kubectl set image puis kubectl rollout undo

Livrable : application accessible, fichiers Terraform et YAML versionnés dans Git.

Quiz niveau Intermédiaire

1 **Quel est l'avantage principal de l'infrastructure as code ?**
A) Reproductibilité et historique des changements B) Se passer de Git C) Remplacer les tests

2 **Pourquoi lire terraform plan avant terraform apply ?**
A) Pour installer le fournisseur B) Pour voir les changements avant de les appliquer C) Pour détruire l'infrastructure

3 **Quelle stratégie bascule d'un coup tout le trafic entre deux environnements ?**
A) Canary B) Rolling update C) Blue / Green

4 **Quels sont les trois piliers de l'observabilité ?**
A) Métriques, logs, traces B) CPU, mémoire, disque C) Build, test, déploiement

5 **Que mesure le taux d'échec des changements (DORA) ?**
A) Le nombre de commits par jour B) La part des changements provoquant un incident ou un retour arrière C) La durée du build

03

GitOps

Git comme source unique de vérité pour l'infrastructure et les applications

Qu'est-ce que GitOps ?

Un mode d'exploitation où l'état désiré du système est décrit de manière déclarative dans Git, et où un agent automatique réconcilie en continu l'état réel avec cet état désiré.

GitOps = IaC + Git + Pull Request + réconciliation automatique



Auditabilité

Chaque changement = un commit



Rollback simple

git revert



Sécurité

Pas d'accès direct au cluster

Les 4 principes GitOps (OpenGitOps)

1 Déclaratif

Le système est décrit par son état désiré, pas par des étapes.

2 Versionné et immuable

L'état désiré est stocké dans Git avec un historique complet.

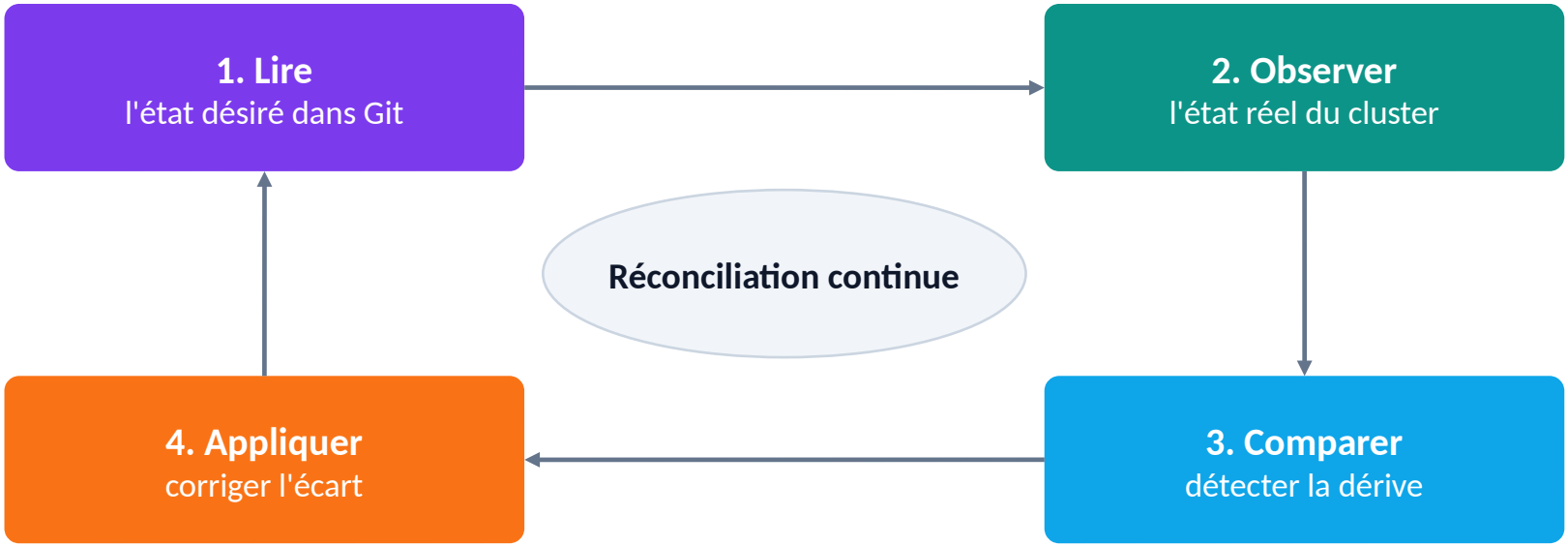
3 Tiré automatiquement

Des agents récupèrent l'état désiré depuis la source.

4 Réconcilié en continu

Les agents corrigent toute dérive entre état réel et désiré.

Schéma : la boucle de réconciliation



Dérive = écart entre Git et le cluster. L'agent la corrige seul (selfHeal), en boucle.

Push (CI/CD classique) vs Pull (GitOps)

PUSH

Git › CI › kubectl › Cluster

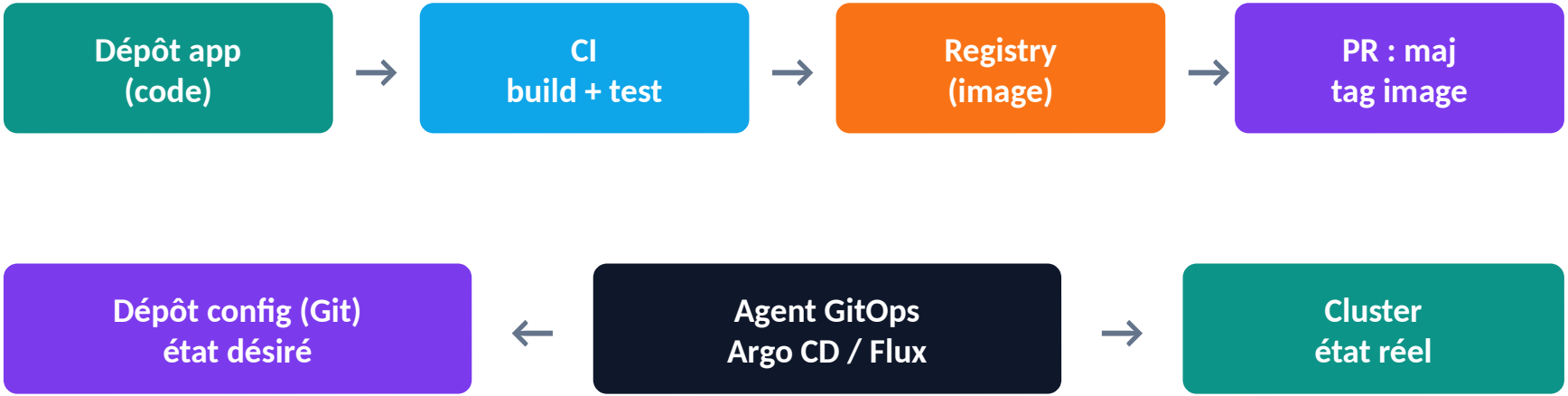
- La CI pousse vers le cluster
- La CI détient les identifiants du cluster
- Dérive non détectée

PULL (GitOps)

Git › Agent › Cluster

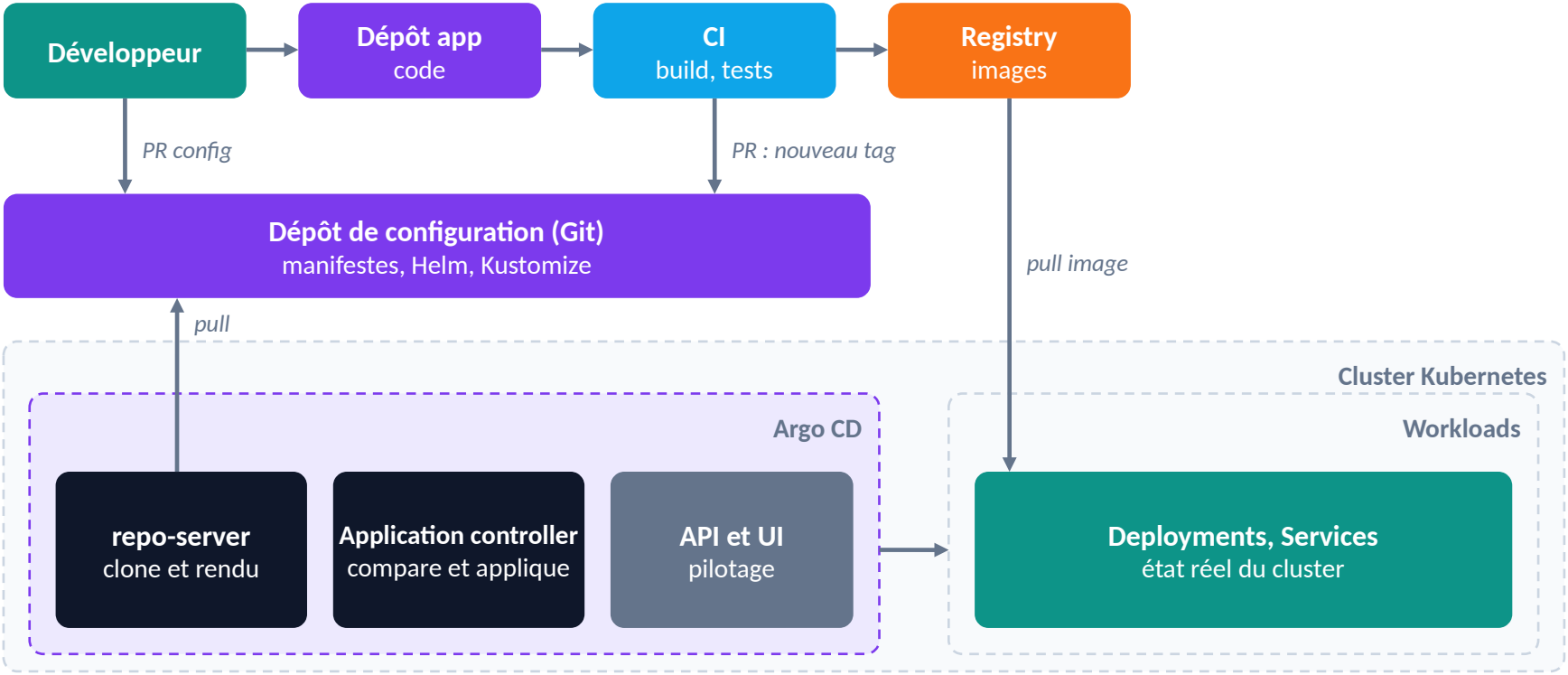
- L'agent tire depuis Git
- Identifiants restent dans le cluster
- Dérive corrigée automatiquement

Architecture GitOps



L'agent compare en continu l'état Git et l'état du cluster, puis réconcilie.

Schéma : architecture GitOps détaillée



Argo CD vs Flux

Critère	Argo CD	Flux
Interface	UI web riche + CLI	CLI ; UI via outils tiers
Architecture	Application unique, ApplicationSet	Contrôleurs modulaires (GitOps Toolkit)
Ressource clé	Application / AppProject	GitRepository / Kustomization / HelmRelease
Multi-cluster	Natif, centralisé	Un Flux par cluster
Images auto	Argo CD Image Updater	Image Automation intégrée
Statut CNCF	Gradué	Gradué

Exemple : une application Argo CD

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: shop
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://git.exemple.com/shop-config.git
    path: overlays/prod
    targetRevision: main
  destination:
    server: https://kubernetes.default.svc
    namespace: shop
  syncPolicy:
    automated: { prune: true, selfHeal: true }
```

source

Où lire l'état désiré

prune

Supprime ce qui n'est plus dans Git

selfHeal

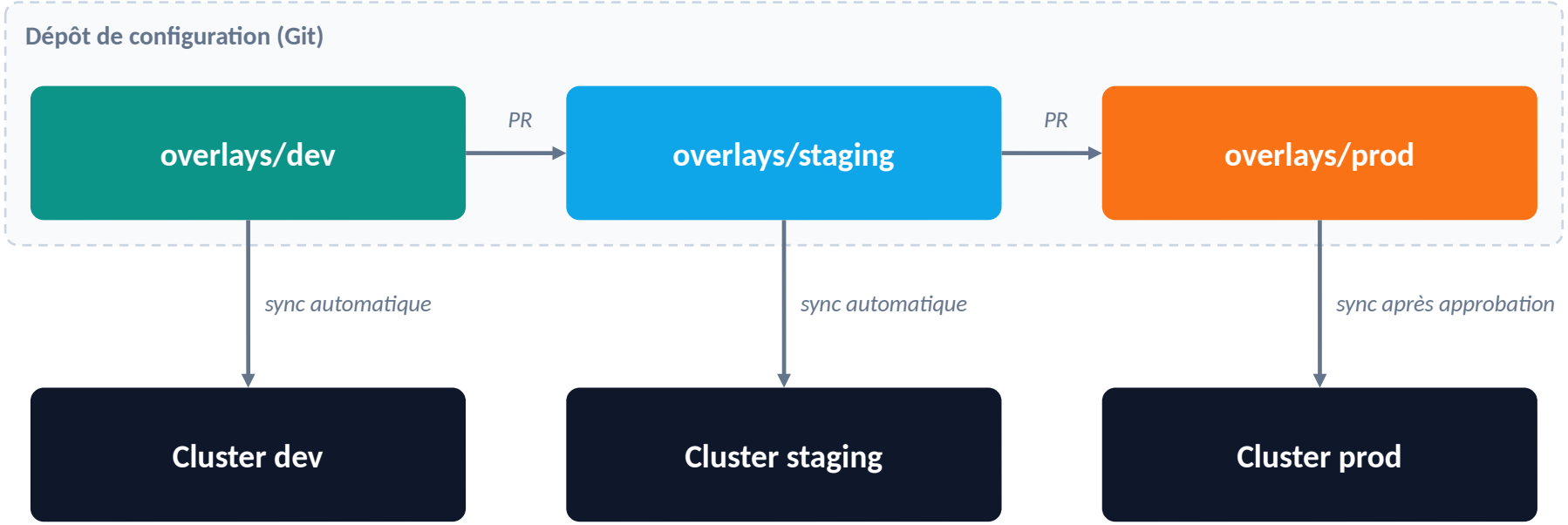
Annule les changements manuels

Organiser ses dépôts et environnements

```
shop-config/
├── base/
│   ├── deployment.yaml
│   └── kustomization.yaml
└── overlays/
    ├── dev/
    ├── staging/
    └── prod/
        ├── kustomization.yaml
        └── patch-replicas.yaml
```

- Dépôt de config séparé du code applicatif
- Un dossier par environnement, pas une branche
- Kustomize : base + overlays (patchs)
- Helm : templates + values par environnement
- Promotion = une PR qui modifie le dossier suivant

Schéma : promotion entre environnements



Promouvoir = ouvrir une pull request qui change le tag d'image ou les valeurs du dossier suivant.

Le workflow GitOps complet

1 · Branche › **2 · Pull Request** › **3 · Revue + CI** › **4 · Merge** › **5 · Sync** › **6 · Vérif.**

Modifier le YAML | Proposer le changement | Tests, policies, approbation | Fusion sur main | L'agent applique | Santé + alertes

Rollback

git revert → l'agent restaure l'état précédent en quelques secondes.

Audit

git log = qui, quoi, quand, pourquoi. Traçabilité native pour la conformité.

DevOps classique vs GitOps

Aspect	DevOps / CD classique	GitOps
Source de vérité	Pipeline + scripts	Git (déclaratif)
Déploiement	Push par la CI	Pull par un agent
Dérive	Non détectée	Détectée et corrigée
Rollback	Relancer un pipeline	git revert
Accès au cluster	CI avec identifiants	Aucun accès externe
Audit	Logs du pipeline	Historique Git

TP : installer Argo CD et déployer par Git

Durée
2 h

Objectif
Déployer une application uniquement par commits Git, avec Argo CD.

Prérequis
Cluster kind, kubectl, dépôt Git de configuration (base + overlays).

1. Installer Argo CD : kubectl create namespace argocd puis kubectl apply -n argocd -f install.yaml
2. Accéder à l'interface : kubectl port-forward svc/argocd-server -n argocd 8080:443
3. Créer une Application pointant sur overlays/dev avec syncPolicy automated
4. Modifier replicas dans Git, faire un commit et observer la synchronisation
5. Rollback : git revert du commit et constat du retour à l'état précédent

Livrable : application synchronisée par Argo CD et un historique Git montrant déploiement et rollback.

TP : dérive, selfHeal et promotion par pull request

Durée
1 h 30

Objectif
Constater la correction automatique de la dérive et promouvoir un changement entre environnements.

Prérequis
TP précédent terminé, deux dossiers overlays/dev et overlays/staging.

1. Activer selfHeal et prune dans la syncPolicy
2. Provoquer une dérive : kubectl scale deployment shop --replicas=5
3. Observer Argo CD ramener le nombre de réplicas à la valeur de Git
4. Créer une branche, modifier l'image dans overlays/staging, ouvrir une pull request
5. Fusionner et vérifier la synchronisation de staging

Livrable : capture de la dérive corrigée et lien vers la pull request de promotion.

Quiz niveau GitOps

1. **Quelle est la source de vérité en GitOps ?**
   A) Le cluster B) Git C) La CI

2. **Dans le modèle pull, qui applique les changements ?**
   A) Un agent situé dans le cluster B) Le développeur depuis son poste C) La CI avec kubectl

3. **Que fait l'option selfHeal d'Argo CD ?**
   A) Supprime les ressources retirées de Git B) Met à jour les images automatiquement C) Annule les modifications manuelles faites sur le cluster

4. **Comment annuler un déploiement en GitOps ?**
   A) Avec kubectl edit B) Avec git revert puis push C) En redémarrant le cluster

5. **Pourquoi préférer des dossiers aux branches par environnement ?**
   A) Les fusions entre branches sont pénibles et provoquent des dérives B) Git n'accepte qu'une seule branche C) Argo CD ne lit que les dossiers

04

Sujets experts

Secrets, multi-cluster, progressive delivery, politiques, supply chain, platform engineering

EXPERT

Gérer les secrets en GitOps

Approche	Principe	À retenir
Sealed Secrets	Chiffrement asymétrique, clé dans le cluster	Simple, secret chiffré dans Git
SOPS + age/KMS	Chiffre les valeurs du YAML	Diff lisible, intégré à Flux
External Secrets	Référence un coffre externe	Rotation centralisée, rien dans Git
Vault / cloud KMS	Coffre-fort source de vérité	Politiques fines, audit

Règle : un secret en clair dans Git est compromis, même après suppression (historique).

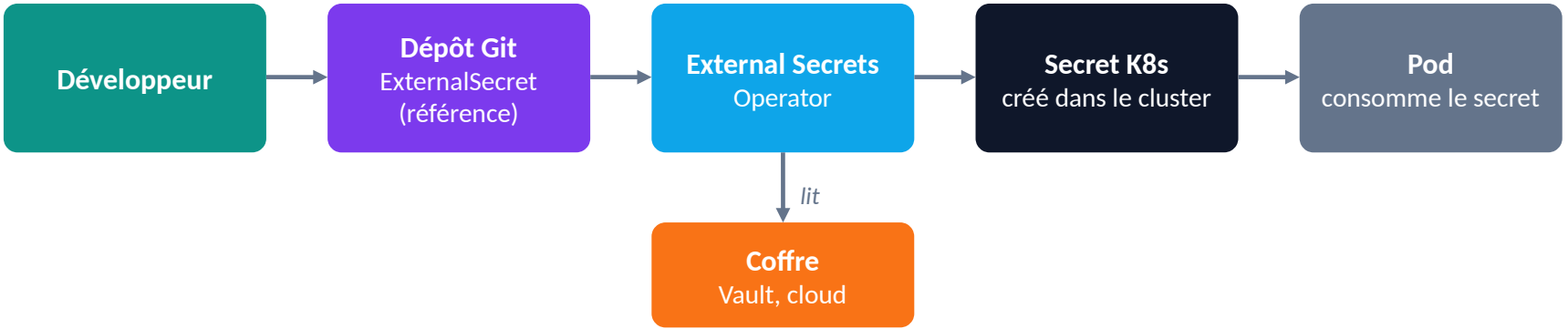
EXPERT

Schéma : gestion des secrets

A. Chiffré dans Git



B. Coffre externe



EXPERT

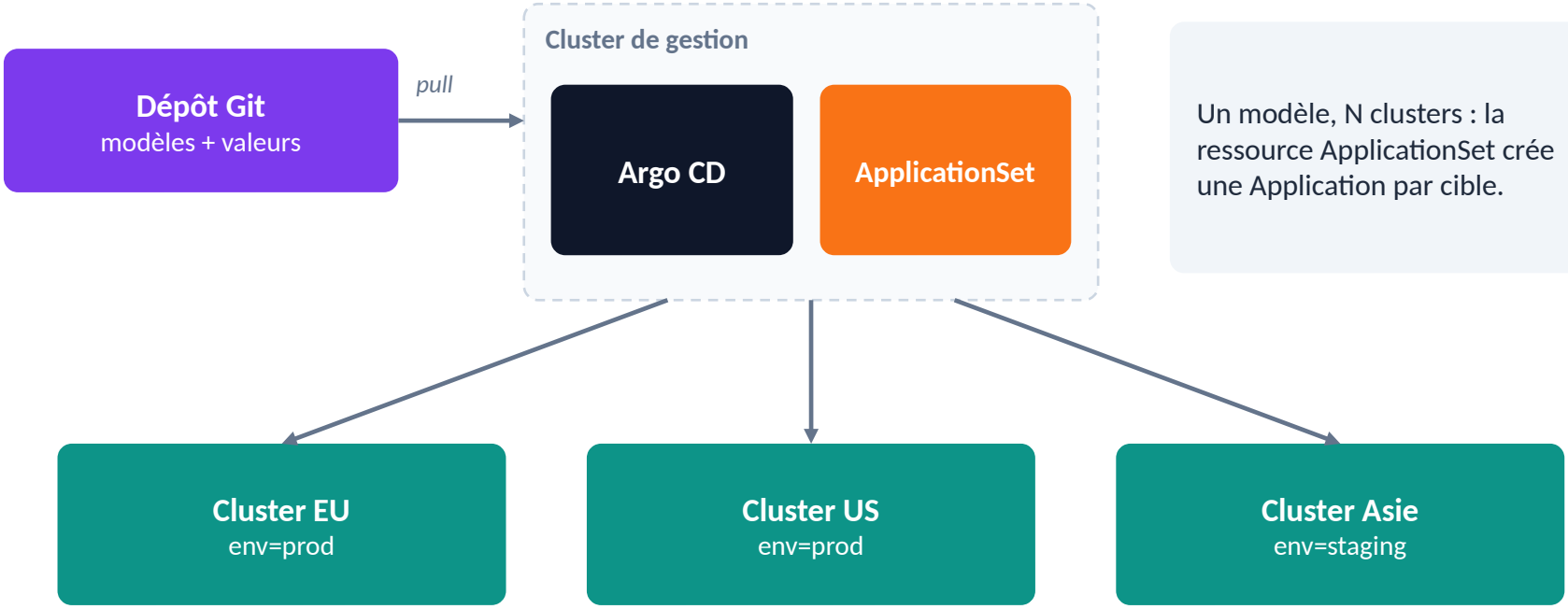
Multi-cluster et ApplicationSets

```yaml
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: shop-clusters
spec:
  generators:
  - clusters: {}
  template:
    metadata:
      name: 'shop-{{name}}'
    spec:
      source:
        repoURL: https://git.exemple.com/shop-config.git
        path: overlays/{{metadata.labels.env}}
      destination:
        server: '{{server}}'
```

- Un modèle, N clusters
- Générateurs : clusters, Git, matrice
- Hub-and-spoke ou un Argo par cluster
- Promotion entre régions par PR

EXPERT

Schéma : multi-cluster hub-and-spoke



Un modèle, N clusters : la ressource ApplicationSet crée une Application par cible.

EXPERT

Progressive delivery

5 % › 25 % › 50 % › 100 %

Analyse automatique des métriques (Prometheus) entre chaque palier

Argo Rollouts

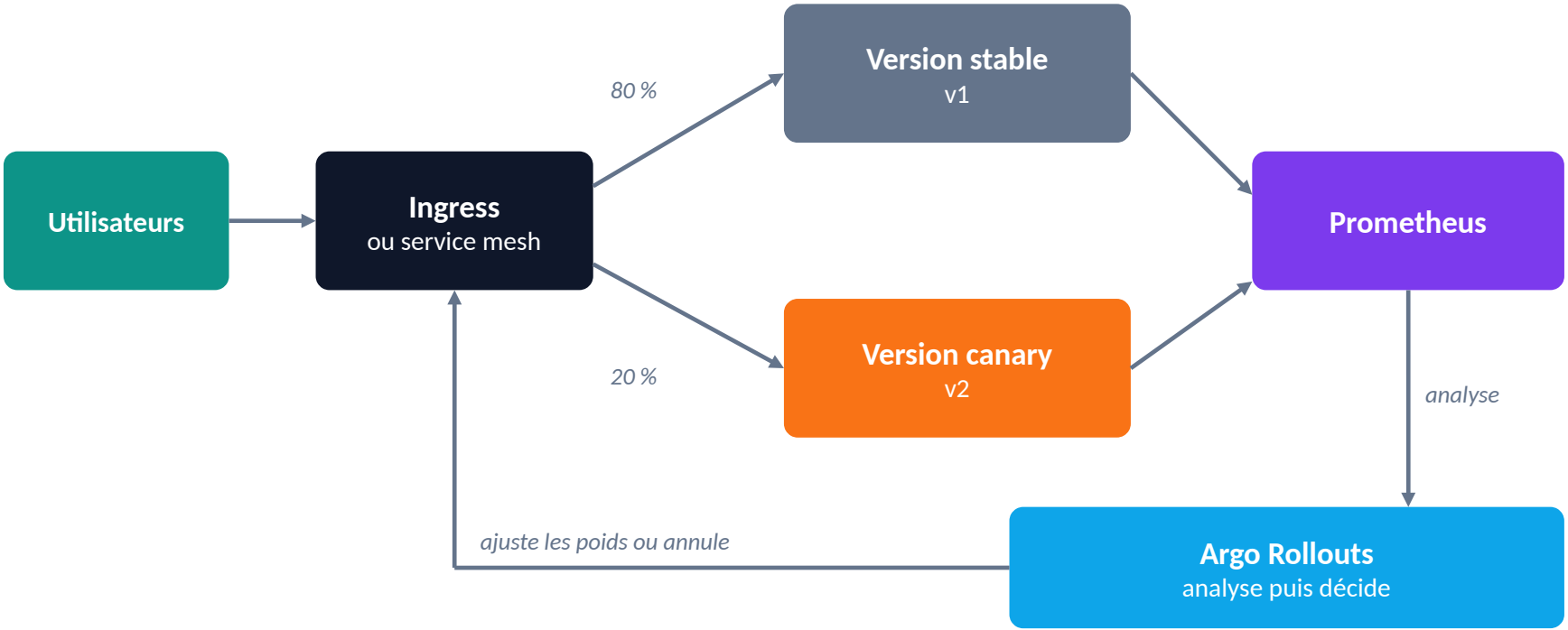
Remplace Deployment par Rollout : canary, blue-green, analyses, intégration ingress / service mesh.

Flagger

Contrôleur de progressive delivery pour Flux, Istio, Linkerd, NGINX.

EXPERT

Schéma : canary piloté par métriques



EXPERT

Policy as Code et sécurité de la chaîne

Policy as Code

OPA Gatekeeper, Kyverno : refuser les conteneurs root, images non signées, absence de limites de ressources.

Signature

Cosign / Sigstore : signer les images, vérifier la signature à l'admission.

SBOM et SLSA

Inventaire des composants (Syft) et niveaux de provenance du build.

EXPERT

Platform Engineering

Réduire la charge cognitive des développeurs avec une plateforme interne en libre-service.



Portail

Backstage : catalogue, docs, templates



Golden paths

Chemins balisés et supportés



GitOps

Moteur de déploiement

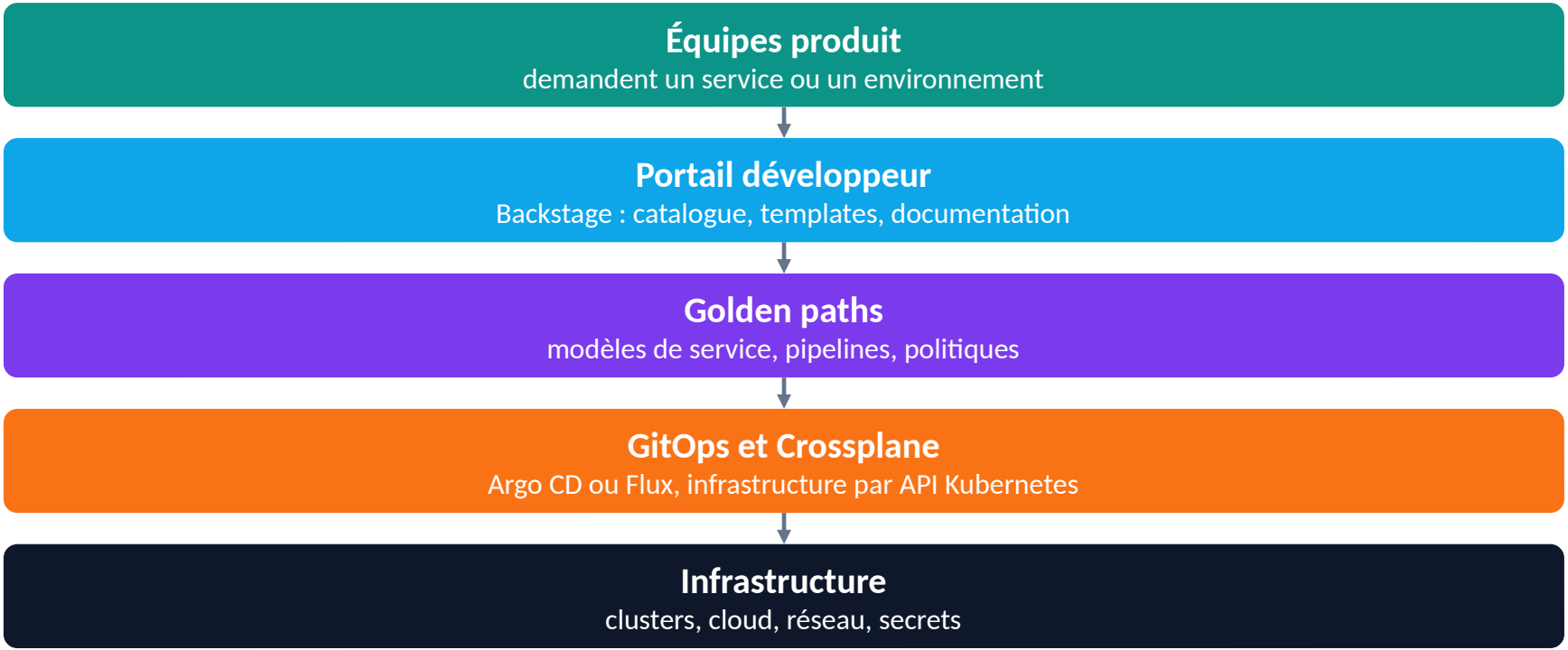


Crossplane

Infra cloud via l'API Kubernetes

EXPERT

Schéma : plateforme interne (IDP)



EXPERT

Pièges et anti-patterns

Anti-pattern	Pourquoi c'est un problème	Alternative
kubectl edit en prod	Dérive, perte de traçabilité	Tout passer par PR + selfHeal
Secrets en clair	Fuite définitive	SOPS, Sealed Secrets, ESO
Une branche par environnement	Merges pénibles, dérives	Dossiers + overlays
Config et code mélangés	Boucles de CI infinies	Dépôt de config séparé
Tag « latest »	Non reproductible	Tags immuables / digests
Sync sans revue	Erreurs en prod	PR obligatoire + protections

EXPERT

TP : secrets, politiques et progressive delivery

Durée
3 h

Objectif
Sécuriser un dépôt GitOps et déployer progressivement avec analyse automatique.

Prérequis
TP GitOps terminés, outils : sops et age, Kyverno, Argo Rollouts.

1. Chiffrer un Secret avec SOPS et age, committer le fichier chiffré, vérifier qu'aucune valeur n'est en clair
2. Installer Kyverno et créer une politique refusant les conteneurs root
3. Tenter de déployer un pod en root et constater le refus à l'admission
4. Convertir le Deployment en Rollout canary (20 %, 50 %, 100 %)
5. Déployer une version défectueuse et observer le rollback automatique

Livrable : dépôt sans secret en clair, politique active et rollout annulé automatiquement.

EXPERT

Quiz niveau Expert

1 **Un secret en clair a été commité puis supprimé. Il est…**
A) Sûr, car supprimé B) Compromis, car présent dans l'historique C) Chiffré automatiquement

2 **Quelle solution référence un coffre externe plutôt que de stocker le secret dans Git ?**
A) External Secrets Operator B) Sealed Secrets C) SOPS

3 **À quoi sert une ApplicationSet ?**
A) Signer des images B) Chiffrer des secrets C) Générer plusieurs Applications à partir d'un modèle

4 **Que fait Argo Rollouts quand l'analyse échoue ?**
A) Rétablit automatiquement la version stable B) Continue jusqu'à 100 % C) Supprime le cluster

5 **Quel outil refuse à l'admission un pod qui tourne en root ?**
A) Trivy B) Kyverno (ou OPA Gatekeeper) C) Cosign

Feuille de route d'apprentissage

Mois 1	Git avancé, Linux, Docker, pipeline GitHub Actions
Mois 2	Terraform, Ansible, Kubernetes, Prometheus / Grafana
Mois 3	Argo CD ou Flux, Kustomize / Helm, secrets, policies
Ensuite	Progressive delivery, multi-cluster, platform engineering. TP : secrets, politiques, canary

Points clés à retenir

DevOps	est avant tout une culture, outillée par l'automatisation
CI/CD	livre vite, souvent, avec fiabilité
IaC	décrit l'infrastructure comme du code
GitOps	Git = vérité, agent = réconciliation continue

Quiz et questions

1. Qui déclenche le déploiement dans un modèle GitOps pull ?

2. Que fait l'option selfHeal d'Argo CD ?

3. Comment annuler un déploiement en GitOps ?

4. Comment stocker un secret sans le mettre en clair dans Git ?

5. Citez les 4 métriques DORA.

Synthèse du cours

Débutant	Intermédiaire	GitOps	Expert
• DevOps = culture + pratiques + outils • Pipeline CI/CD automatisé • Feedback rapide et boucle continue	• IaC, Docker, Kubernetes • Rolling, blue/green, canary • Observabilité, DevSecOps, DORA	• Git = source unique de vérité • Pull + réconciliation continue • Argo CD ou Flux, rollback par revert	• Secrets chiffrés ou externalisés • Multi-cluster, progressive delivery • Policy as code, supply chain, plateforme

Fil rouge : automatiser → versionner → réconcilier

Glossaire (1/2)

Terme	Définition
Blue / Green	Deux environnements identiques, bascule du trafic d'un seul coup
Canary	Nouvelle version envoyée d'abord à une petite part du trafic
CI/CD	Intégration continue et livraison ou déploiement continu
Conteneur	Application et dépendances isolées, exécutées de façon portable
Dérive (drift)	Écart entre l'état réel d'un système et l'état désiré
DevSecOps	DevOps avec la sécurité intégrée à chaque étape
DORA	Quatre métriques de performance de livraison logicielle
GitOps	Exploitation pilotée par Git, avec réconciliation automatique
Helm	Gestionnaire de paquets (charts) pour Kubernetes
IaC	Infrastructure décrite dans des fichiers versionnés

Glossaire (2/2)

Terme	Définition
Image	Modèle immuable à partir duquel démarre un conteneur
Kubernetes	Orchestrateur qui déploie et répare les conteneurs à grande échelle
Kustomize	Personnalisation de manifestes YAML par base et overlays
Observabilité	Comprendre l'état d'un système via métriques, logs et traces
Pipeline	Suite automatisée d'étapes : build, tests, package, déploiement
Pull Request	Proposition de changement soumise à revue avant fusion
Réconciliation	Boucle qui aligne l'état réel sur l'état désiré
Rollback	Retour à une version précédente stable
SBOM	Inventaire des composants logiciels d'un artefact
Shift-left	Détecter les problèmes (sécurité, qualité) le plus tôt possible

A

Corrigés des TP et quiz

Solutions de référence, à consulter après avoir cherché par soi-même

DÉBUTANT

Corrigé TP1 : pipeline CI

```yaml
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npm run lint
      - run: npm test
```

Points clés

- Un test cassé donne un check rouge dans l'onglet Actions
- Corriger, pousser : le check redevient vert
- Badge à coller dans le README : ![CI](https://github.com/ORG/REPO/actions/workflows/ci.yml/badge.svg)
- Protéger main : fusion bloquée si CI rouge

Vérification : le pipeline passe au vert après correction, et le badge s'affiche.

Corrigé TP2 (1/2) : image et Deployment

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
CMD ["node","server.js"]

docker build -t shop:1.0 .
kind create cluster
kind load docker-image shop:1.0
```

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: shop }
spec:
  replicas: 2
  selector:
    matchLabels: { app: shop }
  template:
    metadata:
      labels: { app: shop }
    spec:
      containers:
      - name: shop
        image: shop:1.0
        imagePullPolicy: IfNotPresent
```

Vérification : kubectl get pods affiche 2 pods Running.

Corrigé TP2 (2/2) : Service et Terraform

```
apiVersion: v1
kind: Service
metadata: { name: shop }
spec:
  selector: { app: shop }
  ports: [{ port: 80, targetPort: 3000 }]

kubectl apply -f .
kubectl port-forward svc/shop 8080:80
kubectl set image deploy/shop shop=shop:2.0
kubectl rollout status deploy/shop
kubectl rollout undo deploy/shop
```

```
terraform {
  required_providers {
    docker = {
      source = "kreuzwerker/docker"
    }
  }
}
resource "docker_image" "nginx" {
  name = "nginx:1.27"
}
resource "docker_container" "web" {
  name  = "web"
  image = docker_image.nginx.image_id
  ports { internal = 80  external = 8081 }
}
```

Vérification : http://localhost:8080 répond, et rollout undo restaure l'ancienne version.

Corrigé TP3 (1/2) : installer Argo CD

```
kubectl create namespace argocd
kubectl apply -n argocd \
  -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl -n argocd get pods

kubectl port-forward svc/argocd-server -n argocd 8080:443

kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath="{.data.password}" | base64 -d
```

Vérification : https://localhost:8080 s'ouvre, connexion avec admin et le mot de passe récupéré.

Corrigé TP3 (2/2) : Application et rollback

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: shop
  namespace: argocd
spec:
  source:
    repoURL: https://github.com/ORG/shop-config
    path: overlays/dev
  destination:
    server: https://kubernetes.default.svc
    namespace: shop
  syncPolicy:
    automated: {}
    syncOptions:
      - CreateNamespace=true
```

```
# patch-replicas.yaml
spec:
  replicas: 3

git commit -am "dev: 3 replicas"
git push

# rollback
git revert HEAD --no-edit
git push
```

Vérification : l'interface Argo CD affiche Synced et Healthy après chaque commit.

Corrigé TP4 : dérive et promotion

```
syncPolicy:
  automated:
    prune: true
    selfHeal: true

# provoquer la dérive
kubectl -n shop scale deployment shop \
  --replicas=5
kubectl -n shop get pods -w
# retour à la valeur de Git
```

```
# overlays/staging/kustomization.yaml
resources:
  - ../../base
images:
  - name: shop
    newTag: "1.1"

git switch -c promote-1.1
git commit -am "staging: shop 1.1"
git push -u origin promote-1.1
# PR, revue, merge sur main
```

Vérification : après le scale manuel, le nombre de pods revient à la valeur définie dans Git.

EXPERT

Corrigé TP5 (1/4) : secrets avec SOPS

```
echo key.txt >> .gitignore
age-keygen -o key.txt
sops --encrypt \
  --age $(age-keygen -y key.txt) \
  --encrypted-regex '^(data|stringData)$' \
  secret.yaml > secret.enc.yaml
git add secret.enc.yaml
grep -c "motdepasse" secret.enc.yaml
# affiche 0
SOPS_AGE_KEY_FILE=key.txt \
  sops --decrypt secret.enc.yaml
```

Points clés

- Seules les valeurs sont chiffrées : le diff reste lisible
- Clé privée hors de Git
- Un secret en clair déjà commité est compromis : le renouveler

Vérification : le fichier committé ne contient aucune valeur en clair, et le déchiffrement fonctionne avec la clé.

Corrigé TP5 (2/4) : politique Kyverno

```yaml
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: disallow-root
spec:
  validationFailureAction: Enforce
  rules:
  - name: run-as-non-root
    match: { any: [{ resources: { kinds: [Pod] } }] }
    validate:
      message: "Pas de conteneur root."
      pattern:
        spec:
          securityContext:
            runAsNonRoot: true
```

Test

- kubectl apply -f disallow-root.yaml
- kubectl run root-test --image=nginx
- Refusé à l'admission avec le message de la politique
- Pod conforme : runAsNonRoot: true et image non root

Vérification : le pod en root est refusé, le pod conforme est accepté.

EXPERT

Corrigé TP5 (3/4) : Rollout canary

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata: { name: shop }
spec:
  # replicas, selector, template : idem
  strategy:
    canary:
      steps:
      - setWeight: 20
      - pause: { duration: 30s }
      - setWeight: 50
      - analysis:
          templates:
          - templateName: success-rate
      - setWeight: 100
```

```
kubectl argo rollouts get rollout shop -w

# version défectueuse
kubectl argo rollouts set image shop \
  shop=shop:bad

# analyse en échec :
# statut Degraded, retour à la
# version stable automatique
```

Vérification : la nouvelle version progresse par paliers, puis est annulée si l'analyse échoue.

EXPERT

Corrigé TP5 (4/4) : analyse automatique

```yaml
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata: { name: success-rate }
spec:
  metrics:
  - name: success-rate
    interval: 30s
    failureLimit: 1
    successCondition: result[0] >= 0.95
    provider:
      prometheus:
        address: http://prometheus.monitoring:9090
        query: |
          sum(rate(http_requests_total{status!~"5.."}[1m]))
          / sum(rate(http_requests_total[1m]))
```

Vérification : avec une version défectueuse, la valeur passe sous 0,95 et le rollout est annulé.

Corrigé quiz : Débutant

N°	Réponse	Explication
1	C) Une culture qui rapproche développement et exploitation	DevOps est avant tout une culture de collaboration, outillée par l'automatisation. Ce n'est ni un outil ni un poste.
2	A) Compiler et tester automatiquement chaque commit	Chaque commit déclenche build et tests pour détecter vite les erreurs.
3	B) Le déploiement continu envoie en production sans approbation manuelle	En livraison continue, le passage en production reste une décision manuelle ; en déploiement continu, il est automatique.
4	C) Le corriger avant tout autre travail	Un build rouge s'arrête et se corrige en priorité : c'est la responsabilité de toute l'équipe.
5	A) La séparation entre Dev (changement) et Ops (stabilité)	Les objectifs opposés des deux équipes créent des silos et des livraisons risquées.

Corrigé quiz : Intermédiaire

N°	Réponse	Explication
1	A) Reproductibilité et historique des changements	Les fichiers versionnés donnent le même résultat à chaque application, avec revue et retour arrière.
2	B) Pour voir les changements avant de les appliquer	Le plan liste les créations, modifications et suppressions prévues, avant toute action.
3	C) Blue / Green	Blue/Green maintient deux environnements identiques et bascule le trafic : retour instantané mais coût doublé.
4	A) Métriques, logs, traces	Métriques, logs et traces se complètent pour comprendre pourquoi un système va mal.
5	B) La part des changements provoquant un incident ou un retour arrière	C'est une métrique de stabilité : le pourcentage de déploiements qui dégradent le service.

Corrigé quiz : GitOps

N°	Réponse	Explication
1	B) Git	L'état désiré est décrit dans Git ; le cluster doit y converger.
2	A) Un agent situé dans le cluster	L'agent (Argo CD, Flux) lit Git et applique : aucun identifiant du cluster n'est exposé à l'extérieur.
3	C) Annule les modifications manuelles faites sur le cluster	selfHeal ramène l'état réel à l'état de Git ; c'est prune qui supprime les ressources retirées.
4	B) Avec git revert puis push	Le revert est un nouveau commit que l'agent applique, avec une trace dans l'historique.
5	A) Les fusions entre branches sont pénibles et provoquent des dérives	Une base commune et des overlays par dossier restent lisibles, et la promotion se fait par pull request.

Corrigé quiz : Expert

N°	Réponse	Explication
1	B) Compromis, car présent dans l'historique	Git conserve l'historique : le secret doit être considéré comme fuité et renouvelé.
2	A) External Secrets Operator	Sealed Secrets et SOPS chiffrent dans Git ; External Secrets lit un coffre (Vault, gestionnaire cloud).
3	C) Générer plusieurs Applications à partir d'un modèle	Des générateurs (clusters, dossiers Git) produisent une Application par cible à partir d'un seul modèle.
4	A) Rétablit automatiquement la version stable	Le rollout passe en Degraded et le trafic revient à la version stable.
5	B) Kyverno (ou OPA Gatekeeper)	Kyverno et Gatekeeper appliquent des politiques ; Trivy analyse des images, Cosign les signe.